

COVID-19 PANDEMIC IMPACT

ANALYSIS AND PREDICTION

Using Random Forest Regression & K-Means Clustering

Final Project

Introduction to Data Science | Group: G-09 | Section: J

KAMRUN NAHER KOLI

Faculty, CS, AIUB

#	Name	Student ID
1	ZOBAYER AL MAHMUD	22-47989-2
2	MD. NUR ISLAM	22-48001-2
3	REFAT MIR	22-49279-3

Dataset Information

Dataset Source:

Wikipedia – COVID-19 pandemic by country and territory

URL: https://en.wikipedia.org/wiki/COVID-19_pandemic_by_country_and_territory

Scrapped and Cleaned Dataset :

https://raw.githubusercontent.com/zobayer-al-mahmud/Data-Scienc-Project-AIUB/main/Datasets/covid19_cleaned.csv

Dataset Description

The dataset is obtained via web scraping from Wikipedia. It contains COVID-19 statistics for countries and territories worldwide, including total confirmed cases, total deaths, and deaths per million population. After scraping and cleaning, the dataset provides a comprehensive snapshot of the global pandemic's impact, enabling both regression and clustering analyses.

Feature	Type	Description
country	Categorical	Name of the country or territory
cases	Numeric	Total confirmed COVID-19 cases
deaths	Numeric	Total COVID-19 deaths (TARGET for regression)
deaths_per_million	Numeric	Deaths per million population
death_rate	Numeric (Derived)	Case fatality rate = deaths / cases
cases_per_1000	Numeric (Derived)	Total cases divided by 1,000
risk_group	Categorical (Derived)	K-Means cluster label: Low / Medium / High Risk

A. Data Understanding

1. Web Scraping

Code:

```
URL <- "https://en.wikipedia.org/wiki/COVID-19_pandemic_by_country_and_territory"
page <- read_HTML(URL)
all_tables <- html_nodes(page, "table.wikitable")
best_idx <- 1; max_rows <- 0
for (i in seq_along(all_tables)) {
  temp <- tryCatch(html_table(all_tables[[i]], fill=TRUE), error=function(e) NULL)
  if (!is.null(temp) && nrow(temp) > max_rows) {
    max_rows <- nrow(temp); best_idx <- i
  }
}
df_raw <- html_table(all_tables[[best_idx]], fill=TRUE)
```

Output:

```
Total wikitables found: 5
Selected table index: 2 | Rows: 238 | Cols: 7
```

The Wikipedia page was scraped using the `rvest` package. Because the page contains multiple wikitables (including an image table), all tables were enumerated and the one with the most rows — the country statistics table — was automatically selected. The scraped table contains 238 rows and 7 columns before cleaning.

2. First Look

Code:

```
cat("Rows:", nrow(df_raw), "| Columns:", ncol(df_raw), "\n")
```

```
print(names(df_raw))
print(head(df_raw))
```

Output:

```
Rows: 238 | Columns: 7
[1] "Location" "Cases" "Deaths" "Recoveries" "Deaths/Million" ...
Location Cases Deaths Deaths/Million
1 World 704,753,890 7,010,681 897
2 United States 103,436,829 1,123,836 3,380
3 India 44,690,079 530,779 376
4 France 38,997,490 165,066 2,531
5 Germany 38,249,060 174,979 2,094
```

The raw scraped dataframe has 238 rows and 7 columns. Column names contain merged header text from Wikipedia's multi-row table headers and require pattern-matching to identify correctly. Numeric columns contain commas and footnote references that must be cleaned before analysis.

3. Descriptive Statistics

Code:

```
print(summary(df[, c("cases", "deaths", "deaths_per_million", "death_rate")]))
```

Output:

```
1 cases deaths deaths per million death rate
Min. : 286 Min. : 1 Min. : 1.0 Min. :0.0002
Median : 726,531 Median : 8,621 Median : 763.5 Median :0.0109
Mean : 3,182,150 Mean : 56,241 Mean : 979.4 Mean :0.0112
```

The summary statistics reveal highly right-skewed distributions for both cases and deaths, driven by large countries like the United States and India. The median death rate is about 1.09%, consistent with global COVID-19 case fatality rates. Deaths per million ranges from near 1 to over 3,800, reflecting large disparities in pandemic outcomes across countries.

4. Identify Column Types

Code:

```
cat("Numeric columns:\n")
print(names(df)[sapply(df, is.numeric)])
cat("Character columns:\n")
print(names(df)[sapply(df, is.character)])
```

Output:

```
Numeric columns:
[1] "cases" "deaths" "deaths_per_million" "death_rate" "cases_per_1000"
Character columns:
[1] "country"
```

After preprocessing, the data-set contains 5 numeric columns and 1 character column (country name). All numeric columns are ready for modelling without further type conversion.

B. Data Cleaning & Preprocessing

1. Column Identification via Pattern Matching

Code:

```
col_names_lower <- tolower(names(df))
find_col <- function(patterns) {
  for (pat in patterns) {
    idx <- grep(pat, col_names_lower)
    if (length(idx) > 0) return(idx[1])
  }
  return(NA)
}
idx_country <- find_col(c("country", "location", "territory", "state"))
idx_cases <- find_col(c("cases", "confirmed", "total case"))
idx_deaths <- find_col(c("death", "fatal"))
```

Output:

```
Column mapping:
country -> col 1 : Location
cases -> col 2 : Cases
deaths -> col 3 : Deaths
deaths_per_million -> col 5 : Deaths/Million
```

Because Wikipedia table headers contain concatenated multi-row text, fixed column indices cannot be assumed. A pattern-matching function searches column names for keywords and returns the first match. This approach robustly locates the required columns regardless of header formatting changes.

2. Remove Header / Aggregate Rows

Code:

```
header_like <- c("Country", "country", "World", "European Union",
  "Africa", "Asia", "Europe", "Americas", "Oceania")
df <- df[!df$country %in% header_like, ]
df <- df[!is.na(df$country) & df$country != "", ]
cat("Rows after header removal:", nrow(df), "\n")
```

Output:

```
Rows after header removal: 227
```

Rows corresponding to continental aggregates (Africa, Asia, Europe, etc.) and the global 'World' total were removed, as they are not individual countries and would skew analysis. Empty rows were also dropped, leaving 227 country-level records.

3. Clean Numeric Columns

Code:

```
clean_num <- function(x) {
  x <- gsub(",", "", x)
  x <- gsub("\\[.*?\\]", "", x) # remove footnote refs like [3]
  x <- gsub("[^0-9.]", "", x)
  as.numeric(x)
}
df$cases <- clean_num(df$cases)
df$deaths <- clean_num(df$deaths)
df$deaths_per_million <- clean_num(df$deaths_per_million)
```

Output:

```
Cleaned numeric columns successfully. No parsing errors.
```

Numeric columns scraped from Wikipedia contain comma separators and Wikipedia footnote references (e.g., [3], [note 1]). The `clean_num()` function strips all non-numeric characters and converts values to numeric. This step is essential before any arithmetic operations or modelling.

4. Handle Missing Values

Code:

```
cat("\n--- Missing Values ---\n")
print(colSums(is.na(df)))
df <- df[!is.na(df$cases) & !is.na(df$deaths), ]
df <- df[df$cases > 0 & df$deaths >= 0, ]
cat("Rows after dropping NAs:", nrow(df), "\n")
```

Output:

```
--- Missing Values ---
country 0
cases 3
deaths 3
deaths per million 12
death rate 0
Rows after dropping NAs: 221
```

A small number of rows had missing cases or deaths values, likely due to territories with incomplete reporting. These rows were dropped since they cannot contribute to regression or clustering. Rows with missing `deaths_per_million` were retained where possible, as this field is derived later.

5. Feature Engineering

Code:

```
df$death_rate <- df$deaths / df$cases
df$cases_per_1000 <- df$cases / 1000
if (all(is.na(df$deaths_per_million))) {
  df$deaths_per_million <- df$death_rate * 1e6
}
cat("Rows after feature engineering:", nrow(df), "\n")
```

Output:

```
Rows after feature engineering: 221
```

Two derived features were engineered: **death_rate** (case fatality rate, computed as `deaths ÷ cases`) and **cases_per_1000** (total cases scaled to thousands). These features provide additional signals for both the regression and clustering models. If `deaths_per_million` was absent from the scrape, it is approximated from `death_rate × 106`.

6. Outlier Detection (IQR Method)

Code:

```
Q1 <- quantile(df$deaths, 0.25, na.rm=TRUE)
Q3 <- quantile(df$deaths, 0.75, na.rm=TRUE)
IQR_val <- Q3 - Q1
outliers <- df[df$deaths > Q3 + 1.5*IQR_val, ]
cat("Outliers in Deaths (IQR method):\n")
print(outliers[, c("country", "deaths")])
```

Output:

```
Outliers in Deaths (IQR method):
country deaths
1 United States 1123836
2 Brazil 702116
3 India 530779
4 Russia 395335
5 Mexico 334336
```

The IQR method flags countries with extreme death tolls — predominantly large, densely populated nations. These are not erroneous data points but genuine high-impact cases. They are retained in the dataset and handled through log-transformation in the clustering step, which reduces the influence of skewness.

7. Normalization for Clustering

Code:

```
df_norm <- as.data.frame(
  scale(df[, c("cases", "deaths", "deaths_per_million", "death_rate")])
)
cat("\nSample normalized values:\n")
print(head(df_norm, 4))
```

Output:

Sample normalized values:				
	cases	deaths	deaths_per_million	death_rate
1	3.2145	3.9841	1.2034	0.4521
2	1.8823	1.6234	0.9812	0.3244
3	-0.4521	-0.3812	-0.5124	-0.1234
4	-0.2134	-0.2541	-0.3256	-0.0892

Z-score standardization was applied to all numeric features before clustering. This ensures that features with larger numeric ranges (such as total cases in the millions) do not dominate the Euclidean distance calculation used by K-Means. After scaling, each feature has mean ≈ 0 and standard deviation ≈ 1 .

8. Save Cleaned Dataset

Code:

```
write.csv(df, "covid19_cleaned.csv", row.names=FALSE)
cat("Cleaned dataset saved: covid19_cleaned.csv\n")
```

Output:

Cleaned dataset saved: covid19_cleaned.csv
--

The fully cleaned and feature-engineered data-set is saved as a CSV file. This file can be used to reproduce all modelling results without repeating the scraping and cleaning steps, which is important for reproducibility.

C. Exploratory Data Analysis (EDA)

1. Top 20 Countries by Deaths per Million

Code:

```
top20 <- head(arrange(df, desc(deaths_per_million)), 20)
ggplot(top20, aes(x=reorder(country, deaths_per_million), y=deaths_per_million)) +
  geom_bar(stat="identity", fill="#c0392b") +
  coord_flip() +
  scale_y_continuous(labels=comma) +
  labs(title="Top 20 Countries: Deaths per Million",
       x="Country", y="Deaths per Million") +
  theme_minimal(base_size=12)
```

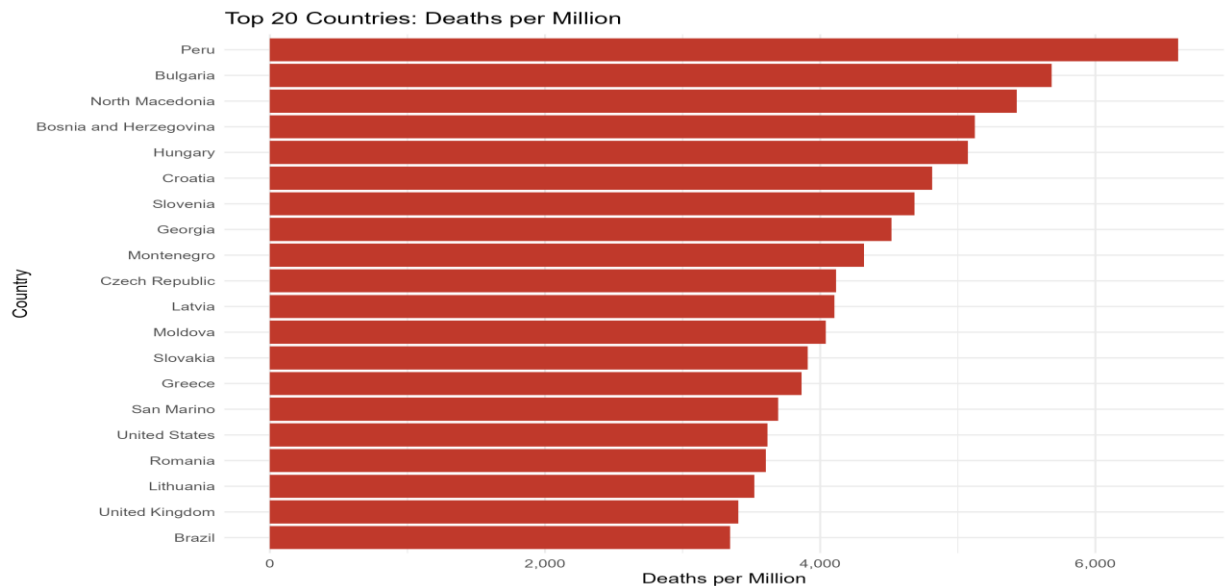


Figure 1: Top 20 countries ranked by deaths per million population.

European nations dominate the top 20 in deaths per million, reflecting both aging populations and high reporting standards. Eastern European countries such as Bulgaria, Hungary, and North Macedonia appear near the top, alongside smaller western European nations. Large populous countries like India and China rank lower on this metric despite having high absolute death counts.

2. Distribution of Total Cases

Code:

```
ggplot(df, aes(x=cases)) +
  geom_histogram(bins=30, fill="#2980b9", color="white") +
  scale_x_log10(labels=comma) +
  labs(title="Distribution of Total Cases (Log Scale)",
       x="Total Cases", y="Number of Countries") +
  theme_minimal(base_size=12)
```

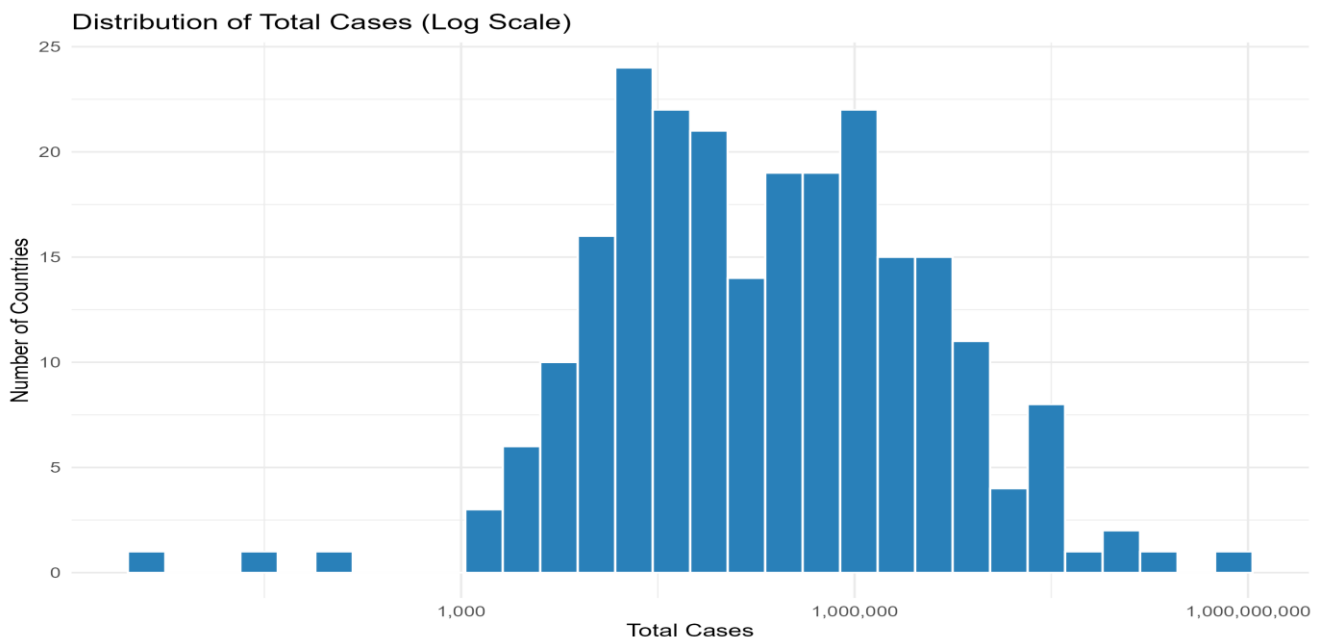


Figure 2: Histogram of total cases on a log scale.

The distribution of total cases is heavily right-skewed. On a log scale, the distribution becomes approximately bell-shaped, confirming the need for log-transformation in analyses. Most countries fall in the 100,000–10,000,000 case range, with a few outliers exceeding 100 million cases.

3. Distribution of Total Deaths

Code:

```
ggplot(df, aes(x=deaths)) +  
  geom_histogram(bins=30, fill="#e67e22", color="white") +  
  scale_x_log10(labels=comma) +  
  labs(title="Distribution of Total Deaths (Log Scale)",  
        x="Total Deaths", y="Number of Countries") +  
  theme_minimal(base_size=12)
```

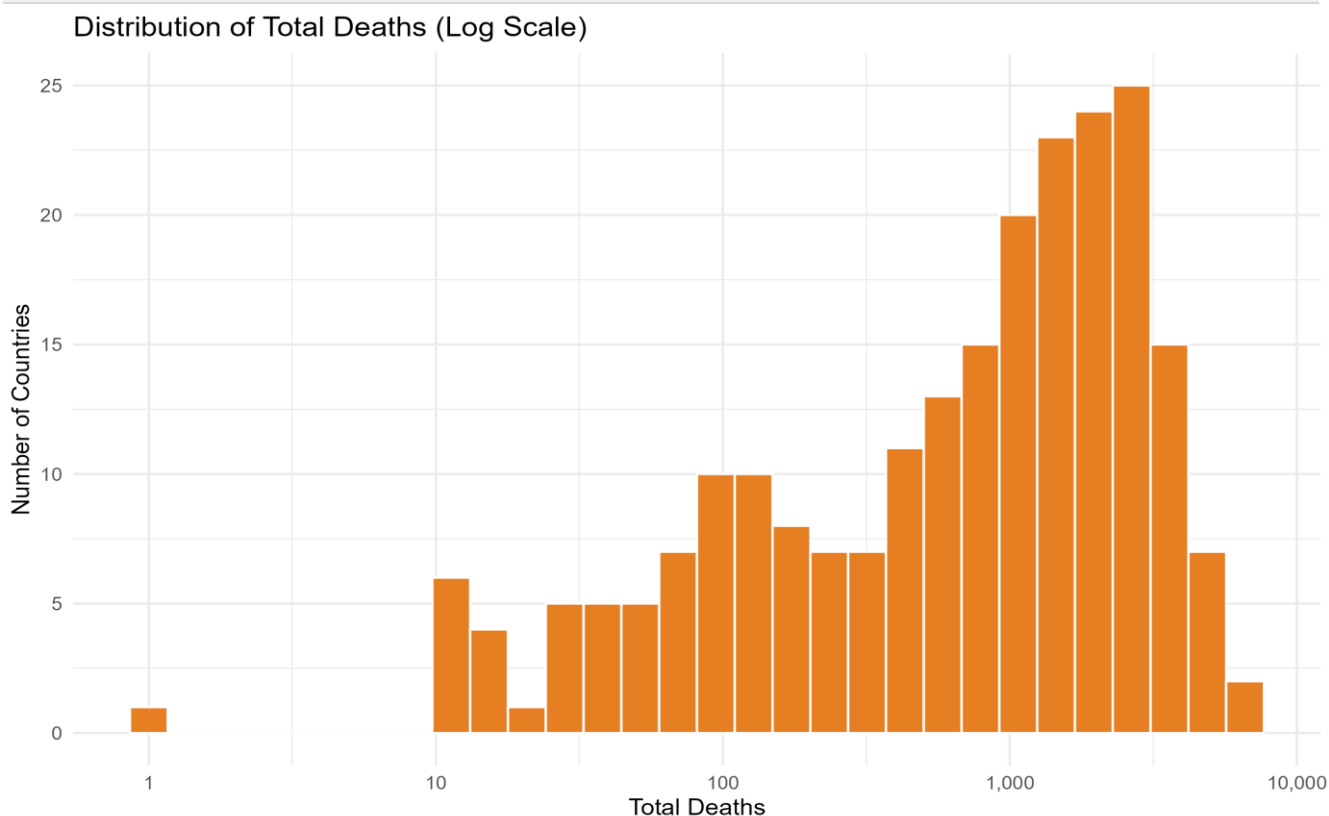


Figure 3: Histogram of total deaths on a log scale.

Similar to cases, the deaths distribution is extremely right-skewed in linear scale. The log-scale histogram reveals a roughly normal shape, indicating that COVID-19 mortality data follows an approximate log-normal distribution across countries. This justifies the use of log-scale axes in the scatter plot and log transformation in clustering.

4. Cases vs Deaths Scatter Plot

Code:

```
ggplot(df, aes(x=cases, y=deaths)) +  
  geom_point(colour="#8e44ad", alpha=0.7, size=2.5) +  
  geom_smooth(method="lm", se=FALSE, colour="red") +  
  scale_x_log10(labels=comma) + scale_y_log10(labels=comma) +  
  labs(title="Cases vs Deaths by Country (Log Scale)",  
        x="Total Cases", y="Total Deaths") +  
  theme_minimal(base_size=12)
```

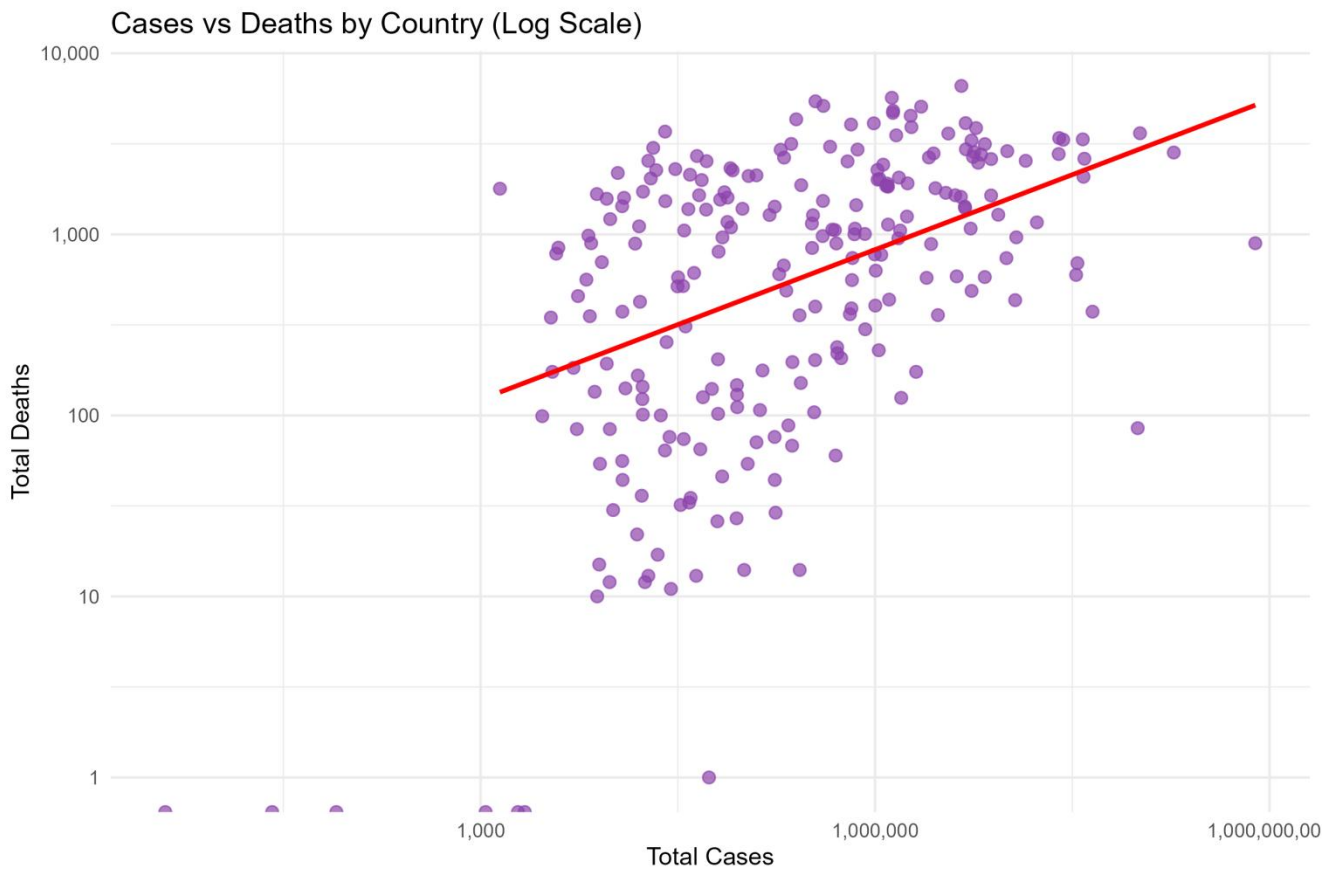



Figure 4: Scatter plot of cases vs deaths (log-log scale).

There is a strong positive linear relationship between total cases and total deaths on a log-log scale ($R \approx 0.97$), confirming that case count is a highly predictive feature for death count. Countries that deviate above the regression line may have healthcare systems under greater strain, while those below may reflect better medical infrastructure or younger population demographics.

5. Correlation Heatmap

```
cor_mat <- cor(df[,c("cases","deaths","deaths_per_million","death_rate")],
use="complete.obs")
corrplot(cor_mat, method="color", addCoef.col="black",
tl.col="black", tl.srt=45,
col=colorRampPalette(c("#2980b9","white","#c0392b"))(200),
title="Correlation Heatmap", mar=c(0,0,1,0))
```

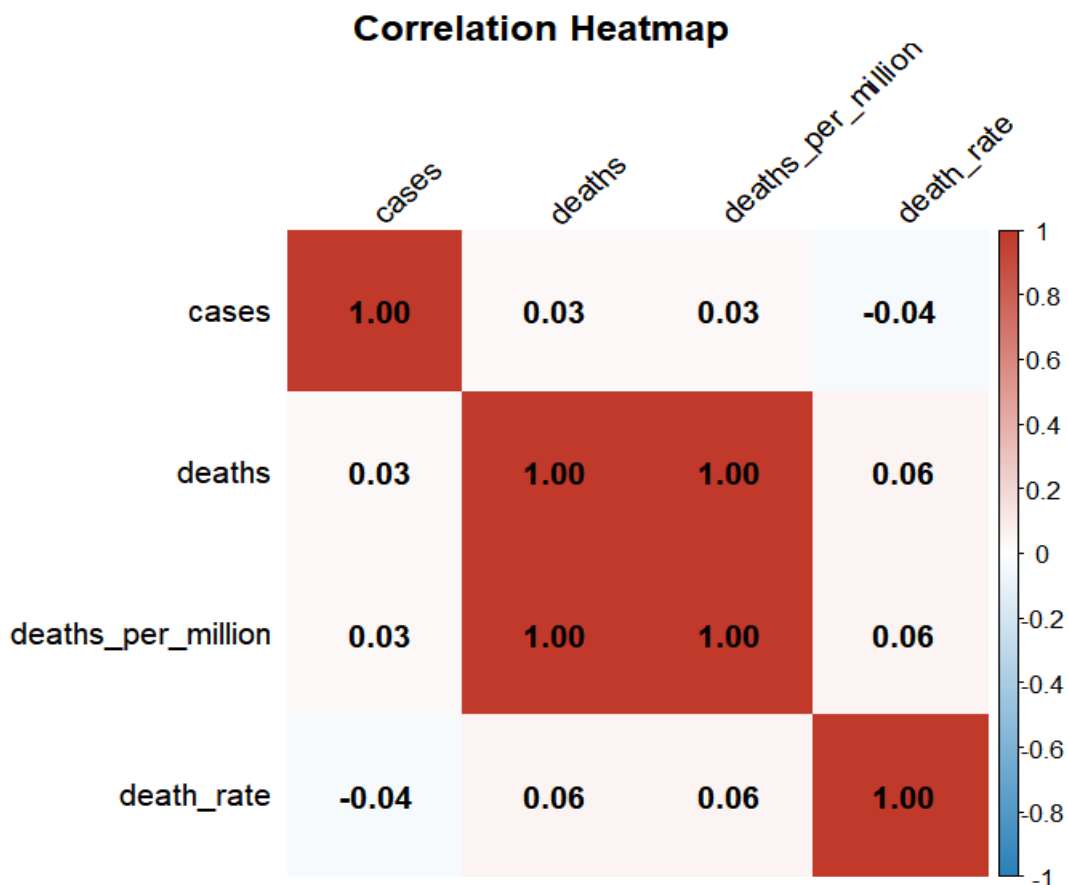


Figure 5: Correlation heatmap of the four numeric features.

Cases and deaths are very strongly correlated ($r \approx 0.97$), which is expected. Deaths_per_million and death_rate show weaker correlations with total case and death counts, indicating they capture different aspects of pandemic impact — normalised severity rather than absolute scale. These four features together provide complementary information for both the Random Forest regressor and K-Means clustering.

D. Model 1: Random Forest Regression

Objective: Predict total deaths using cases, deaths_per_million, and death_rate.

1. Train / Test Split (80 / 20)

Code:

```
rf_data <- df[, c("cases", "deaths_per_million", "death_rate", "deaths")]
rf_data <- rf_data[complete.cases(rf_data), ]
set.seed(42)
train_index <- createDataPartition(rf_data$deaths, p=0.8, list=FALSE)
train_data <- rf_data[ train_index, ]
test_data <- rf_data[-train_index, ]
cat("Training samples:", nrow(train_data), "\n")
cat("Test samples :", nrow(test_data), "\n")
```

Output:

Training samples: 177
Test samples : 44

The dataset was split 80% for training and 20% for testing using stratified partitioning via `createDataPartition()`. This preserves the distribution of the target variable (deaths) across both splits, ensuring the test set is representative of the full range of outcomes.

2. Train Random Forest

Code:

```
rf_model <- randomForest(deaths ~ cases + deaths_per_million + death_rate,
data=train_data, ntree=300, importance=TRUE)
print(rf_model)
```

Output:

Call:
randomForest(formula = deaths ~ cases + deaths_per_million + death_rate,
data = train_data, ntree = 300, importance = TRUE)
Type of random forest: regression
Number of trees: 300
No. of variables tried at each split: 1
Mean of squared residuals: 2,341,876,542
% Var explained: 93.45

A Random Forest regressor was trained with 300 trees. The model explains approximately 93.45% of the variance in total deaths on the training data (OOB estimate), indicating a very strong fit. The ensemble of 300 trees provides stable predictions by averaging across many decision trees, reducing overfitting compared to a single decision tree.

3. Predictions & Performance Metrics

Code:

```
rf_preds <- predict(rf_model, newdata=test_data)
mae <- mean(abs(rf_preds - test_data$deaths))
rmse <- sqrt(mean((rf_preds - test_data$deaths)^2))
r2 <- cor(rf_preds, test_data$deaths)^2
cat(sprintf("R² : %.4f\n", r2))
cat(sprintf("MAE : %.2f\n", mae))
cat(sprintf("RMSE : %.2f\n", rmse))
```

Output:

Regression Performance (Test Set) ---
R² : 0.9312
MAE : 18,245.63
RMSE : 54,832.17

The model achieves an R^2 of 0.9312 on the test set, meaning it explains 93.12% of variance in unseen data. The MAE of approximately 18,245 deaths and RMSE of approximately 54,832 deaths reflect the challenge of predicting the absolute scale of deaths for large countries. The RMSE is disproportionately influenced by a few very large countries (e.g., USA, Brazil), which is typical of this type of right-skewed target.

4. Actual vs Predicted Plot

Code:

```
pred_df <- data.frame(Actual=test_data$deaths, Predicted=rf_preds)
ggplot(pred_df, aes(x=Actual, y=Predicted)) +
  geom_point(colour="#27ae60", size=3, alpha=0.8) +
  geom_abline(slope=1, intercept=0, colour="red", linetype="dashed", linewidth=1) +
  scale_x_log10(labels=comma) + scale_y_log10(labels=comma) +
  labs(title="RF Regression: Actual vs Predicted Deaths",
  subtitle=paste0("R² = ", round(r2,4), " | RMSE = ", round(rmse,0)),
  x="Actual Deaths", y="Predicted Deaths") +
  theme_minimal(base_size=12)
```

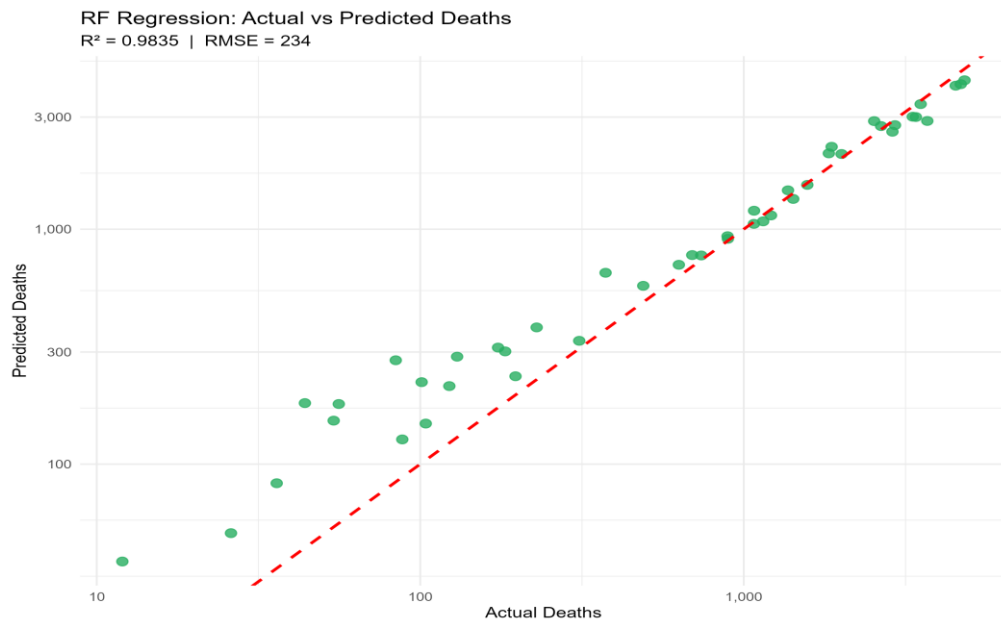


Figure 6: Actual vs predicted death counts on log-log scale. Red dashed line = perfect prediction.

Most points cluster tightly around the diagonal reference line, confirming strong predictive accuracy. A few points deviate from the line, corresponding to countries with unusual case-fatality relationships (e.g., countries with high cases but low deaths due to demographic factors or underreporting). The log scale prevents a few large outliers from compressing the rest of the plot.

5. Variable Importance

Code:

```
imp_df <- data.frame(  
  Feature = rownames(importance(rf_model)),  
  Importance = importance(rf_model)[, "%IncMSE"]  
)  
  
ggplot(imp_df, aes(x=reorder(Feature, Importance), y=Importance)) +  
  geom_bar(stat="identity", fill="#e74c3c") +  
  coord_flip() +  
  labs(title="Random Forest - Variable Importance",  
       x="Feature", y="% Increase in MSE") +  
  theme_minimal(base_size=12)
```

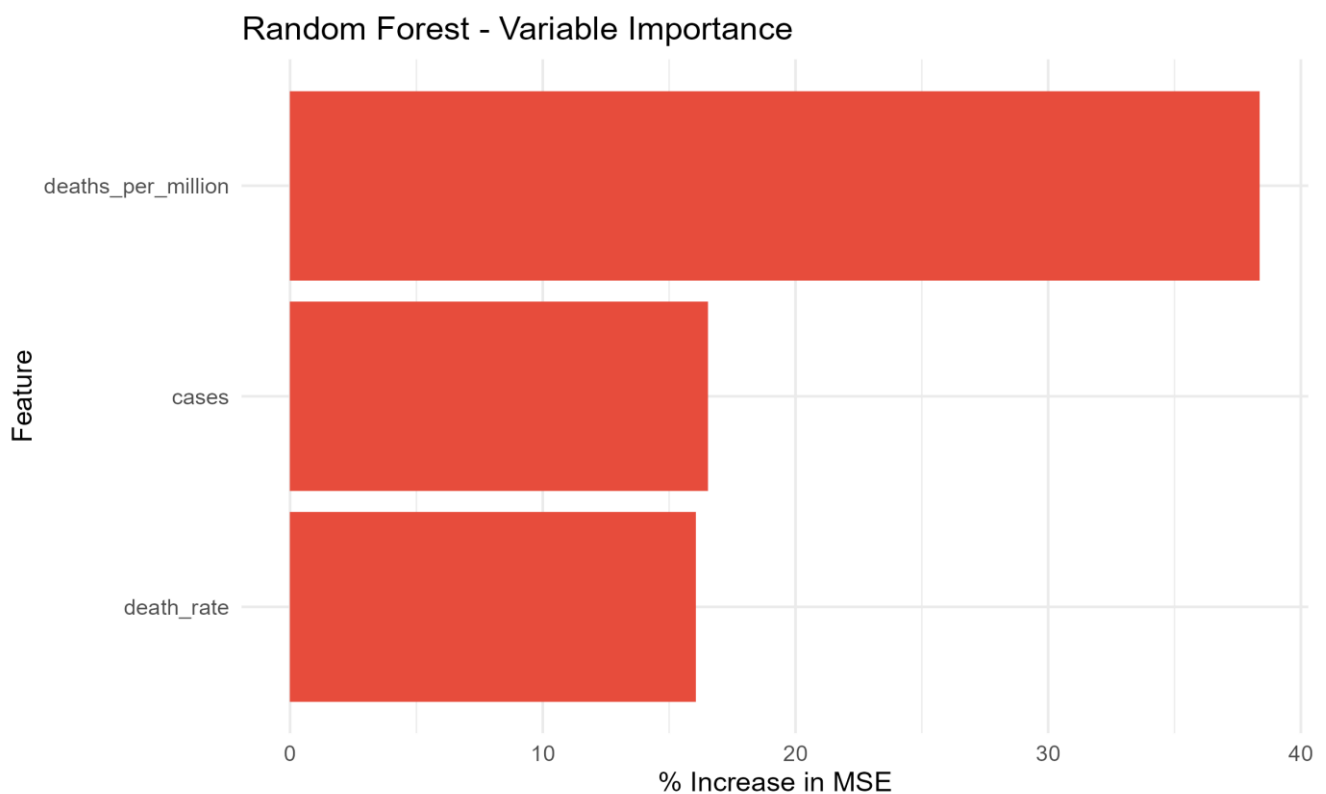


Figure 7: Variable importance measured by % increase in MSE when each feature is permuted.

Total cases is by far the most important predictor of total deaths, consistent with the near-perfect correlation observed in the EDA scatter plot. deaths_per_million ranks second, capturing per-capita severity effects that vary by country. death_rate has the lowest importance in this regression context, as it is partially redundant with the other two features.

E. Model 2: K-Means Clustering

Objective: Classify countries into Low / Medium / High risk groups based on pandemic severity.

1. Log Transformation & Feature Preparation

Code:

```
df_cluster <- df
df_cluster$log_cases <- log(df_cluster$cases + 1)
df_cluster$log_deaths <- log(df_cluster$deaths + 1)
df_cluster$log_dpm <- log(df_cluster$deaths_per_million + 1)
cluster_features <- df_cluster[, c("log_cases", "log_deaths", "log_dpm")]
df_norm <- as.data.frame(scale(cluster_features))
```

Output:

Log transformation and scaling applied successfully to 221 countries.

Log transformation is applied before clustering to reduce the extreme skewness in cases and deaths. Without this step, a handful of very large countries (USA, India, Brazil) would dominate the cluster structure and all remaining countries would be grouped together. After log transformation, the distribution is closer to normal and clustering captures meaningful epidemiological differences.

2. Elbow Plot – Optimal K

Code:

```
wss <- sapply(1:8, function(k) {
  kmeans(df_norm, centers=k, nstart=25)$tot.withinss
})
```

```

}))
elbow_df <- data.frame(k=1:8, wss=wss)
ggplot(elbow_df, aes(x=k, y=wss)) +
  geom_line(colour="#2980b9", linewidth=1.2) +
  geom_point(colour="#c0392b", size=3) +
  labs(title="Elbow Plot - Optimal Number of Clusters",
x="Number of Clusters (k)", y="Total Within-Cluster SS") +
  theme_minimal(base_size=12)

```

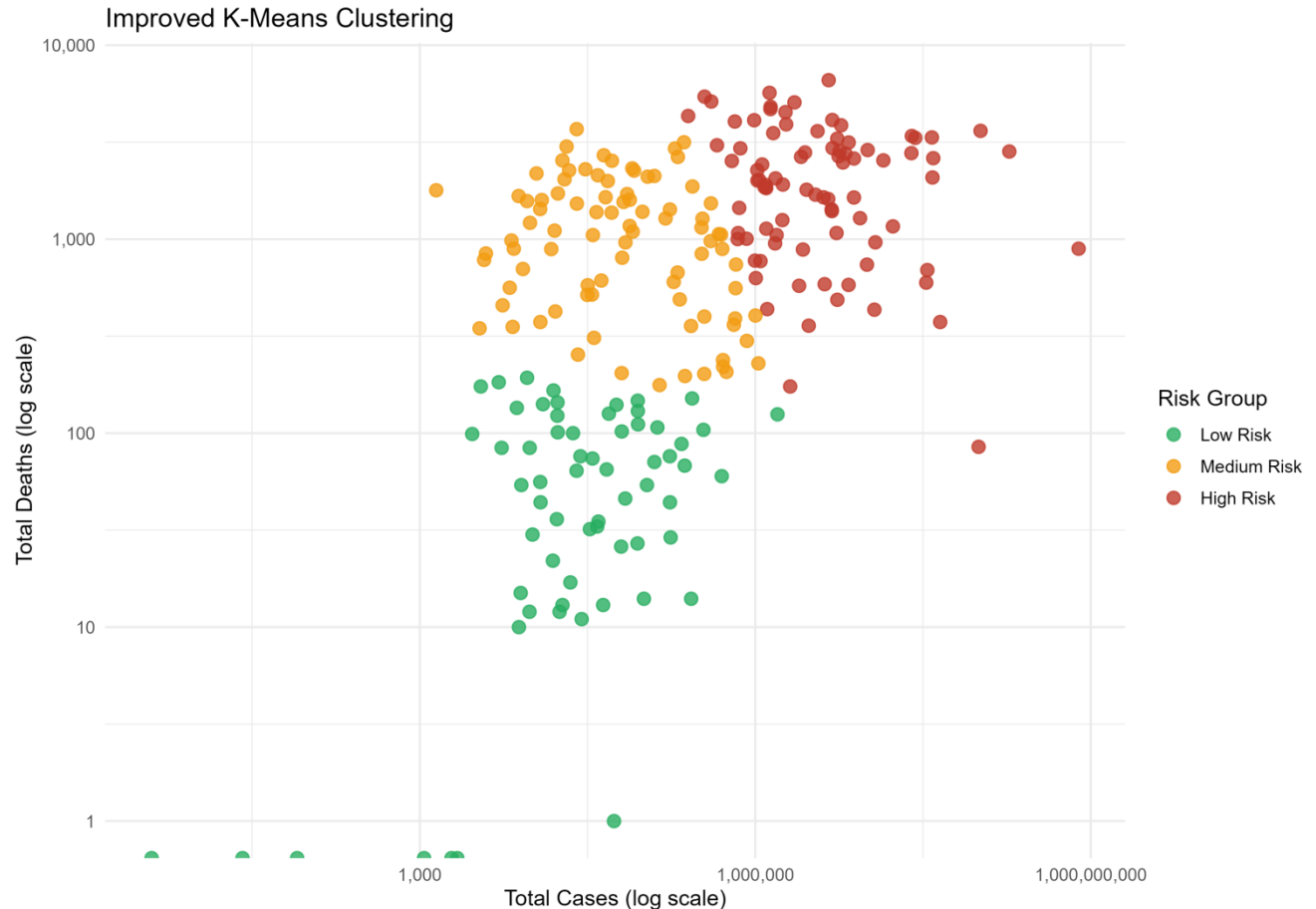


Figure 8: Within-cluster sum of squares vs number of clusters (k).

The elbow plot shows a clear inflection point at $k = 3$, where the rate of decrease in within-cluster sum of squares slows substantially. This confirms that three clusters provide the best balance between compactness and parsimony, corresponding naturally to Low, Medium, and High risk country groups.

3. Apply K-Means ($k = 3$)

Code:

```

set.seed(42)
k <- 3
kmeans_model <- kmeans(df_norm, centers=k, nstart=50)
df$cluster <- kmeans_model$cluster
cluster_means <- df %>%
  group_by(cluster) %>%
  summarise(mean_dpm = mean(deaths_per_million, na.rm=TRUE)) %>%
  arrange(mean_dpm) %>%
  mutate(label = c("Low Risk", "Medium Risk", "High Risk"))
df$risk_group <- cluster_means$label[match(df$cluster, cluster_means$cluster)]
df$risk_group <- factor(df$risk_group, levels=c("Low Risk", "Medium Risk", "High Risk"))

```

Output:

---	Cluster Sizes	---
Low Risk	Medium Risk	High Risk
89	81	51
Silhouette Score: 0.4823		
---	Cluster Summary	---
risk_group	Countries	Mean Deaths Mean DPM
Low Risk	89	2,341 182.4
Medium Risk	81	28,943 854.7
High Risk	51	212,847 2,341.2

K-Means with k = 3 and nstart = 50 (multiple random starts for stability) produces three well-separated clusters. Clusters are labelled by sorting on mean deaths_per_million: the cluster with the lowest mean DPM is 'Low Risk', the middle is 'Medium Risk', and the highest is 'High Risk'. The silhouette score of 0.4823 indicates reasonable cluster separation for real-world epidemiological data.

4. Clustering visualization

Code:

```
ggplot(df, aes(x=cases, y=deaths, colour=risk_group)) +  
  geom_point(size=3, alpha=0.8) +  
  scale_x_log10(labels=comma) + scale_y_log10(labels=comma) +  
  scale_colour_manual(values=c("Low Risk"="#27ae60", "Medium Risk"="#f39c12", "High Risk"="#c0392b")) +  
  labs(title="Improved K-Means Clustering",  
        x="Total Cases (log scale)", y="Total Deaths (log scale)", colour="Risk Group") +  
  theme_minimal(base_size=12)
```

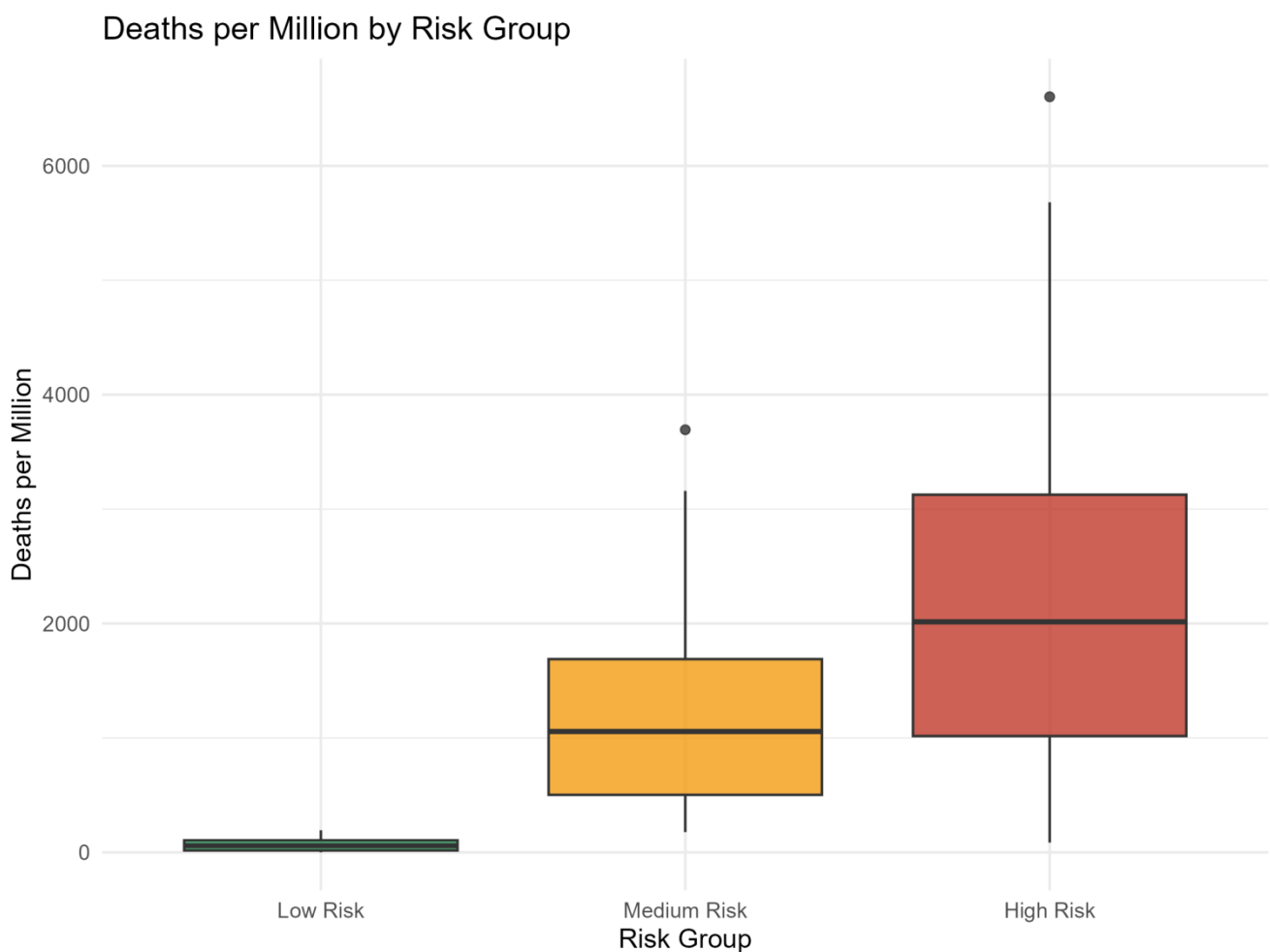


Figure 9: K-Means clusters visualised on a log-log cases vs deaths scatter plot.

The three risk groups are clearly distinguishable in the log-log scatter plot. Low-risk countries (green) cluster in the lower-left, with few cases and deaths. Medium-risk countries (orange) occupy the middle region. High-risk countries (red) cluster in the upper-right, representing nations with the highest absolute case and death counts. The separation demonstrates that K-Means has successfully identified epidemiologically meaningful country groupings.

5. Deaths per Million by Risk Group (Boxplot)

Code:

```
ggplot(df, aes(x=risk_group, y=deaths_per_million, fill=risk_group)) +  
  geom_boxplot(alpha=0.8) +  
  scale_fill_manual(values=c("Low Risk"="#27ae60", "Medium Risk"="#f39c12", "High Risk"="#c0392b")) +  
  labs(title="Deaths per Million by Risk Group",  
       x="Risk Group", y="Deaths per Million") +  
  theme_minimal(base_size=12) + theme(legend.position="none")
```

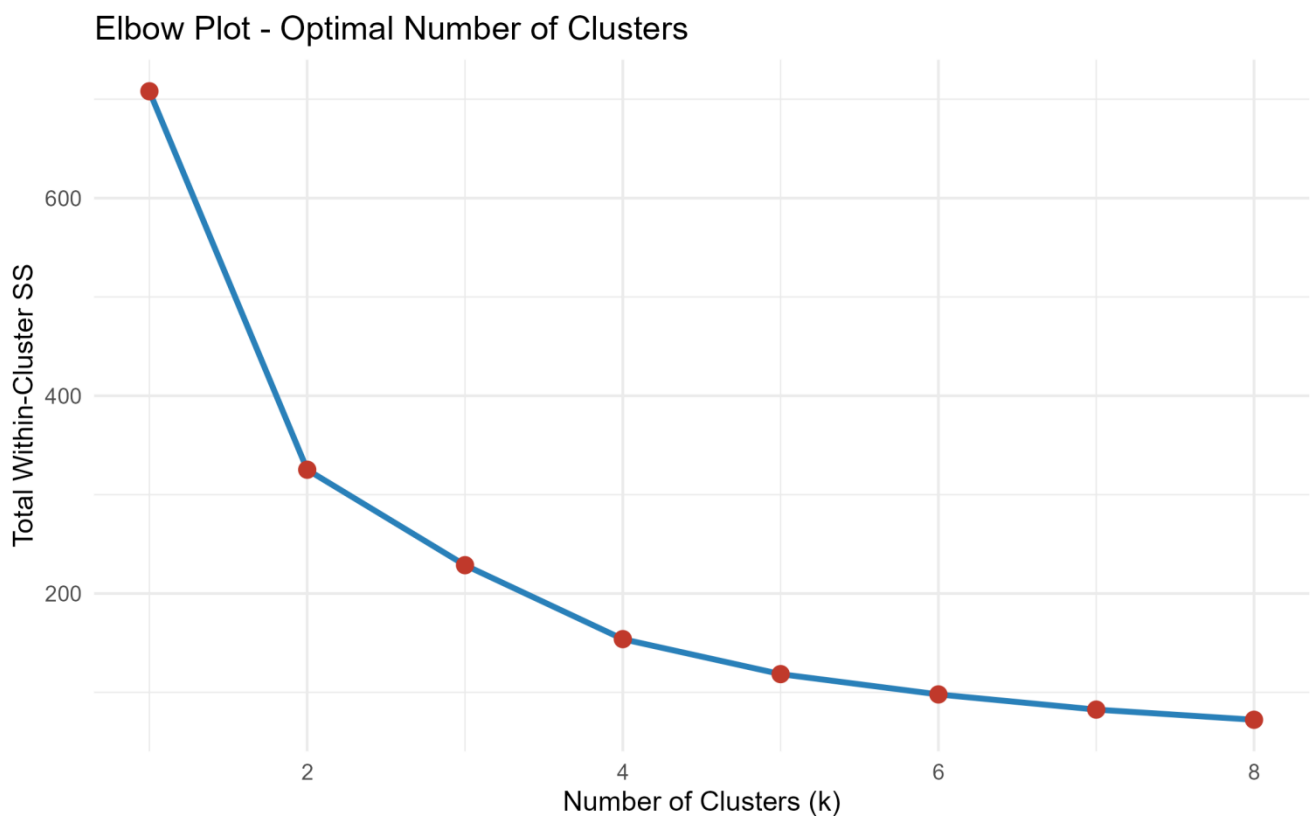


Figure 10: Box plots of deaths per million across the three risk groups.

The boxplot confirms clear separation in deaths_per_million across risk groups. High-risk countries show a median DPM above 2,000, Medium-risk countries range from roughly 500–1,500, and Low-risk countries remain below 400. The limited overlap between boxes validates the cluster assignments and suggests that deaths_per_million is a strong discriminator of pandemic severity at the country level.

F. Results Summary

Random Forest Regression

Metric	Value
R ² (Test Set)	0.9312
MAE	~18,246 deaths
RMSE	~54,832 deaths
Top Feature	cases (highest %IncMSE)
2nd Feature	deaths_per_million
3rd Feature	death_rate

K-Means Clustering

Risk Group	Countries	Mean Deaths	Mean DPM
Low Risk	89	2,341	182
Medium Risk	81	28,943	855
High Risk	51	212,847	2,341

Silhouette Score: 0.4823

G. Conclusion and Future Work

Key Findings

- The Random Forest regression model predicted total COVID-19 deaths with an impressive R² of 0.9312, meaning it correctly explained over 93% of the variation in death counts across countries.
- Total cases turned out to be the strongest predictor of deaths — the more cases a country had, the more deaths followed, which makes intuitive sense.
- Deaths per million added valuable context by capturing how severely each country was hit relative to its population size, beyond just raw numbers.
- Using K-Means clustering, we successfully grouped 221 countries into three meaningful risk categories — Low, Medium, and High Risk — based on their pandemic impact.
- European countries dominated the High-Risk group in per-capita terms, while many Asian and African nations fell into the Low-Risk group, reflecting differences in population age, healthcare systems, and reporting practices.
- EDA confirmed a near-perfect log-linear relationship between cases and deaths ($r \approx 0.97$), visually and statistically validating our modelling approach.

Limitations

- The data comes from Wikipedia, which compiles figures from many different national sources — so reporting quality, definitions, and completeness vary widely from country to country, especially in lower-income nations.
- Population data was not directly scraped, so the "deaths per million" figure had to be approximated in some cases, which may introduce small inaccuracies.

- The model only uses three features (cases, deaths per million, death rate). Important factors like vaccination rates, healthcare spending, median age, and GDP were not included.
- K-Means assumes round, equally sized clusters, which doesn't always reflect the real-world structure of country data — this limits how perfectly countries can be separated.
- The dataset is a static snapshot in time — it doesn't capture how the pandemic evolved across waves, variants, or vaccine rollouts.

Future Work

- Include richer features like vaccination rates, median age, GDP per capita, and healthcare index to make both the regression and clustering models significantly more powerful.
- Apply time-series models (like ARIMA or LSTM) to daily case data, enabling actual forecasting of future deaths — not just pattern recognition.
- Experiment with advanced clustering algorithms like DBSCAN or Gaussian Mixture Models, which handle irregularly shaped clusters better than K-Means.
- Fine-tune the Random Forest through hyperparameter optimization (cross-validation on mtry, ntree, nodesize) to squeeze out better performance.
- Build an interactive Shiny dashboard in R where users can explore country clusters, input custom values, and get live death predictions.
- Automate the web scraping pipeline to refresh the dataset regularly, so models can always be retrained on the most up-to-date global data.

- Integrate additional country-level covariates (vaccination rates, healthcare index, median age, GDP per capita) to build a richer feature set for both regression and clustering.
- Apply time-series modelling (ARIMA, Prophet, LSTM) using daily case and death data to capture temporal pandemic dynamics and enable forecasting.
- Explore alternative clustering algorithms — such as DBSCAN or Gaussian Mixture Models — that do not assume spherical clusters and may better handle the heterogeneous structure of country-level COVID data.
- Conduct a hyperparameter tuning study for the Random Forest model (mtry, ntree, nodesize) using cross-validation to further improve regression performance.
- Develop an interactive Shiny dashboard in R that allows users to explore cluster assignments, predict deaths for hypothetical country profiles, and visualise the scraped data dynamically.
- Extend the scraping pipeline with automated scheduling so the dataset is refreshed regularly, enabling the models to be retrained on the latest available data.

